

OE-EC704A — Web Technology

Semester VII • 3L:0T:0P • 3 Credits

Module 4 Exam Notes: Exception Handling, Strings, I/O Streams & Collections

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1	Java Exception Handling Mechanism	2
1.1	Checked vs. Unchecked Exceptions	2
2	Java String Handling	3
2.1	String vs. StringBuffer	3
2.2	String and StringBuffer Constructors	3
2.3	Core String Operations	4
2.4	Utility Conversions: <code>toString()</code> and <code>valueOf()</code>	4
3	Java Input/Output (I/O) Streams	5
3.1	Byte Stream vs. Character Stream	5
4	Java Collections Framework	5
4.1	ArrayList vs. LinkedList	6

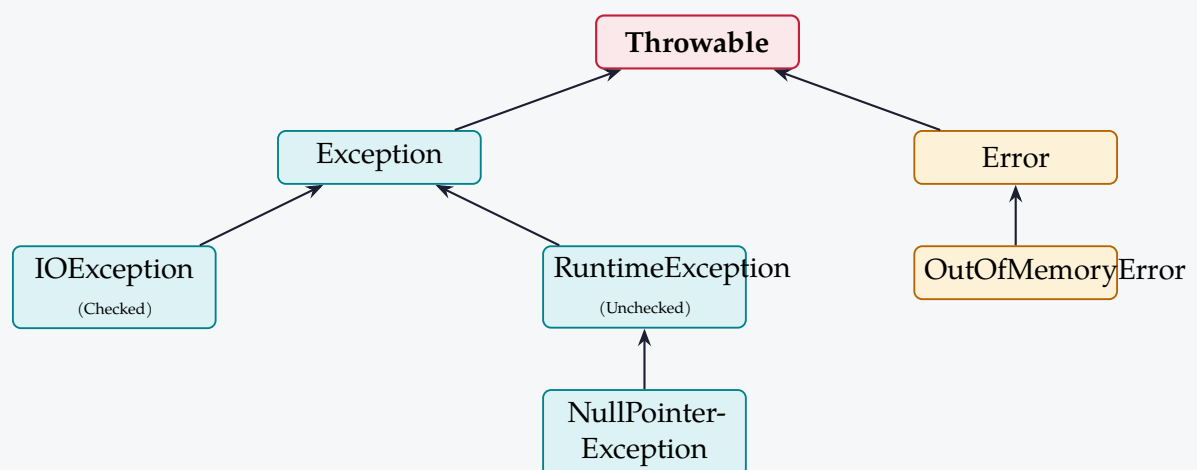
1. Java Exception Handling Mechanism

An exception is an event that disrupts the normal flow of instruction execution. Java uses a structured exception handling system based on five keywords: `try`, `catch`, `throw`, `throws`, and `finally`.

Exception Types

- **Checked Exceptions (Compile-time):** Exceptions that must be declared in the method signature or handled in a try-catch block. Verified by the compiler (e.g., `IOException`, `SQLException`).
- **Unchecked Exceptions (Runtime):** Exceptions that extend `RuntimeException`. Not verified by the compiler (e.g., `NullPointerException`, `ArithmeticException`).
- **Errors:** Serious conditions that applications should not try to catch. They represent system failures (e.g., `OutOfMemoryError`, `StackOverflowError`).

Java Exception Class Hierarchy



1.1 Checked vs. Unchecked Exceptions

Feature	Checked Exceptions	Unchecked Exceptions
Compiler Checks	Enforced at compilation.	Not verified by the compiler.
Base Superclass	Inherits from <code>java.lang.Exception</code> .	Inherits from <code>java.lang.RuntimeException</code> .
Resolution Rule	Must use <code>try-catch</code> or declare <code>throws</code> .	Recommended to write defensive checks.
Examples	<code>FileNotFoundException</code> , <code>IOException</code>	<code>ArithmeticException</code> , <code>NullPointerException</code>

2. Java String Handling

Java represents sequences of characters using the `String` class.

String Immutability

Java `String` objects are **immutable**, meaning their state cannot be modified after creation.

- **String Pool:** When a string literal is declared, the JVM checks if it exists in the String Constant Pool (SCP) on the heap. If present, it reuses the reference, saving heap memory.
- **Why Immutable?** Ensures thread safety, security (parameters like DB connection URLs cannot be altered), and enables string pool caching.

2.1 String vs. StringBuffer

Metric	String Class	StringBuffer Class
Mutability	Immutable (read-only once created).	Mutable (can append or modify content).
Performance	Slower for frequent concatenations (creates new objects).	Fast and memory efficient (modifies local buffer).
Thread Safety	Naturally thread-safe due to immutability.	Thread-safe; methods are synchronized.
Heap Allocation	SCP or standard heap space.	Standard heap space only.

2.2 String and StringBuffer Constructors

Both classes provide multiple constructors to instantiate character sequences:

Constructor	Signature	Description
Default String	<code>String()</code>	Initializes a newly created, empty <code>String</code> object.
Char Array String	<code>String(char[] value)</code>	Allocates a new <code>String</code> representing the char array's contents.
String Copier	<code>String(String original)</code>	Initializes a copy of the argument string.
Buffer Converter	<code>String(StringBuffer sb)</code>	Converts a mutable <code>StringBuffer</code> into an immutable <code>String</code> .
Default Buffer	<code>StringBuffer()</code>	Creates an empty buffer with an initial capacity of 16 characters.
Capacity Buffer	<code>StringBuffer(int cap)</code>	Creates an empty buffer with the specified initial capacity.
String Buffer	<code>StringBuffer(String str)</code>	Creates a buffer initialized with the specified string content.

2.3 Core String Operations

The `String` class provides a rich set of methods for manipulating character sequences. Because `String` is immutable, all modifying methods return a **new** `String` instance representing the result.

Operation	Key Method Signature	Functional Description
Extraction	<code>char charAt(int index)</code>	Returns the char value at the specified index (0 to length – 1).
	<code>int length()</code>	Returns the number of characters in the string.
Comparison	<code>boolean equals(Object obj)</code>	Compares string values for character-by-character equality.
	<code>boolean equalsIgnoreCase(String s)</code>	Value comparison ignoring case differences.
	<code>int compareTo(String anotherString)</code>	Lexicographically compares two strings (returns 0 if equal, negative if less, positive if greater).
Searching	<code>int indexOf(String str)</code>	Returns the index of the first occurrence of the substring (or –1).
	<code>int lastIndexOf(String str)</code>	Returns the index of the last occurrence of the substring.
	<code>boolean contains(CharSequence s)</code>	Returns true if the string contains the specified sequence.
Modifying	<code>String substring(int begin, int end)</code>	Extracts a new substring from <code>begin</code> (inclusive) to <code>end</code> (exclusive).
	<code>String concat(String str)</code>	Appends the specified string to the end (equivalent to the <code>+</code> operator).
	<code>String replace(char old, char new)</code>	Returns a new string with all occurrences of <code>old</code> replaced by <code>new</code> .
	<code>String trim()</code>	Returns a copy with leading and trailing whitespaces omitted.

2.4 Utility Conversions: `toString()` and `valueOf()`

Java provides built-in mechanisms to convert objects and primitive values into string representations:

- **The `toString()` Method:** Defined in the root `Object` class. Every class inherits it, and standard library classes (like `String`, `StringBuffer`, `ArrayList`) override it to return a readable value representation.
 - When an object reference is printed via `System.out.println(obj)`, the JVM implicitly invokes `obj.toString()`.
 - By default, it returns: `getClass().getName() + "@" + Integer.toHexString(hashCode())`.

- **The Static `String.valueOf()` Methods:** Overloaded static utility methods that convert any primitive type or object into a String representation.
 - `String.valueOf(int i)` returns a string representing the integer.
 - `String.valueOf(boolean b)` returns "true" or "false".
 - `String.valueOf(Object obj)` returns `obj.toString()` or "null" if the object reference is null (preventing a `NullPointerException`).

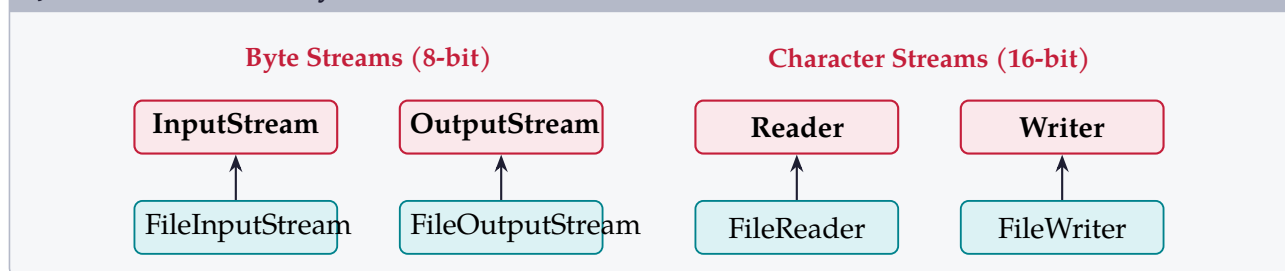
3. Java Input/Output (I/O) Streams

An I/O stream represents an abstraction of a data source or destination (e.g., keyboard, file, network socket).

3.1 Byte Stream vs. Character Stream

Metric	Byte Stream	Character Stream
Data Unit	Processes data as raw 8-bit bytes.	Processes data as 16-bit Unicode characters.
Use Case	Binary files (images, audio, zip files).	Text files (documents, configuration text).
Base Classes	<code>InputStream</code> and <code>OutputStream</code>	<code>Reader</code> and <code>Writer</code>
Key Classes	<code>FileInputStream</code> , <code>FileOutputStream</code>	<code>FileReader</code> , <code>FileWriter</code>

Java Stream Hierarchy



4. Java Collections Framework

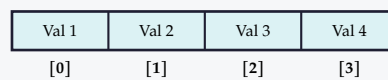
The Collections Framework provides a unified architecture for storing and manipulating groups of objects.

4.1 ArrayList vs. LinkedList

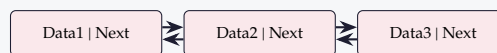
Metric	ArrayList	LinkedList
Internal Structure	Dynamically resizing array.	Doubly linked list.
Access Time	$O(1)$ random access (index-based).	$O(n)$ search access (sequential traverse).
Insertion/Deletion	$O(n)$ (requires element shifts).	$O(1)$ if pointer is located.
Memory Overhead	Low (only stores elements).	Higher (stores element and prev/next links).

ArrayList vs. LinkedList Memory Layout

ArrayList (Contiguous Memory):



LinkedList (Pointer Links):



Quick Revision — Module 4

Exception Handling, Strings, I/O Streams & Collections | OE-EC704A

Exception, File I/O & Collections Syntax

Concept	Syntax and Example Usage
Try-Catch Block	<pre>try { int a = 5/0; } catch(ArithmeticException e) { ... }</pre>
Throws Declaration	<pre>void readFile() throws IOException { ... }</pre>
Custom Exception	<pre>class MyException extends Exception { MyException(String s){super(s);} }</pre>
Buffer Reader I/O	<pre>BufferedReader br = new BufferedReader(new InputStreamReader(System.in));</pre>
Collections & Strings	Core Declarations & Queries
ArrayList Declare	<pre>ArrayList<String> list = new ArrayList<String>();</pre>
Collection Iterator	<pre>Iterator<String> itr = list.iterator(); while(itr.hasNext()) { ... }</pre>
String Concat	<pre>String s = "Biraj" + " Sarkar"; // SCP optimization</pre>
StringBuffer Append	<pre>StringBuffer sb = new StringBuffer("CGE"); sb.append("C");</pre>

One-Line Definitions

- **Checked Exception:** An exception subclass checked during compilation that requires explicit handling.
- **Unchecked Exception:** RuntimeException subclasses resulting from logical design faults.
- **Immutability:** State constraint preventing modification of heap values after initial allocation.
- **Stream:** Abstract communication pipe that handles sequential transfer of data bytes.
- **Byte Stream:** Stream subclass designed to process binary data using 8-bit units.
- **Character Stream:** Stream subclass designed to process Unicode text data using 16-bit units.
- **Collections Framework:** Framework of interfaces and classes for managing structures.
- **Iterator:** Universal collection interface cursor that iterates elements in a forward direction.

Frequently Asked / Viva Questions**Q1. What is the difference between `final`, `finally`, and `finalize`?**

`final` is a modifier indicating that a class cannot be inherited, a method cannot be overridden, or a variable cannot be reassigned. `finally` is a block accompanying a try-catch block to execute clean-up operations. `finalize` is a method invoked by the Garbage Collector before reclaiming an object.

Q2. Can we write a try block without a catch block? If so, how?

Yes, a try block can be followed by a `finally` block instead of a catch block. Additionally, Java 7 supports the **try-with-resources** statement, which automatically closes resources without requiring an explicit catch or finally.

Q3. What is try-with-resources statement?

It is a try block declared with one or more resources (e.g., streams). The resources must implement `java.lang.AutoCloseable`. They are automatically closed when exiting the try block.

Q4. Why are String objects immutable while StringBuffer objects are mutable?

Strings are cached in the String Constant Pool to save memory and ensure thread safety. `StringBuffer` is designed for dynamic character manipulation without creating multiple garbage references.

Q5. What is the difference between `equals()` and `==` when comparing strings?

`==` compares reference equality (whether they point to the exact same object reference). The `equals()` method compares value equality (whether the character sequences are identical).

Q6. What is the difference between byte stream classes and character stream classes?

Byte stream classes read and write raw 8-bit bytes (binary), whereas character stream classes read and write 16-bit Unicode characters, handling text encoding automatically.

Q7. Why do we wrap a `FileReader` inside a `BufferedReader`?

Wrapping inside a `BufferedReader` reduces disk I/O requests by reading data blocks into an internal buffer. It also provides the convenient `readLine()` method.

Q8. What is the difference between `ArrayList` and `Vector`?

`ArrayList` is non-synchronized, fast, and not thread-safe. `Vector` is synchronized, has thread-safe methods, and carries a significant performance overhead.

Q9. Why does a `LinkedList` consume more memory than an `ArrayList`?

An `ArrayList` stores elements contiguously inside a backing array. A `LinkedList` stores elements inside node wrappers that also hold memory addresses for the previous and next nodes.

Q10. How does the `throws` keyword differ from the `throw` keyword?

`throw` is used to explicitly throw a single exception instance inside a method block. `throws` is used in a method declaration to list possible checked exceptions.

Comparison: Try-Catch-Finally vs. Try-With-Resources

Metric	Standard Try-Catch-Finally	Try-With-Resources (Java 7+)
Resource Closing	Manual (must write stream close calls in finally).	Automatic (handled by compiler behind the scenes).
Boilerplate Code	High (requires nested try-catch blocks in finally).	Minimal (declared inside try parameters).
Exception Suppression	Primary exception can be lost if close fails.	Retains primary exception; close errors suppressed.
Target Classes	Any standard classes.	Only classes implementing <code>AutoCloseable</code> .